MAT700 Mathematical Methods for Data Mining - Lecture4

A. Balinsky

Cardiff School of Mathematics

Distance Measures and Measures of Similarity

We now take a short detour to study the general notion of **distance measures**.

Definition (Distance Measure)

Suppose we have a set of points, called a *space*. A *distance measure* on this space is a function $d(x, y)$ that takes two points in the space as arguments and produces a real number, and satisfies the following axioms:

1. $d(x, y) \geq 0$ (no negative distances).
2. $d(x, y) = 0$ if and only if $x = y$ (distances are positive, except for the distance from a point to itself).
3. $d(x, y) = d(y, x)$ (distance is symmetric).
4. $d(x, y) \leq d(x, z) + d(z, y)$ (the triangle inequality).

Example (Distances)

- **Euclidean Distances or L_2-norm**:

$$d([x_1, x_2, \ldots, x_n], [y_1, y_2, \ldots, y_n]) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

- **L_r-norm ($r \geq 1$)**:

$$d([x_1, x_2, \ldots, x_n], [y_1, y_2, \ldots, y_n]) = \left(\sum_{i=1}^{n} |x_i - y_i|^r \right)^{1/r}.$$

L_1-norm: *Manhattan distance*.
L_∞-norm: is defined as the maximum of $|x_i - y_i|$ over all dimensions i.

Example (Distances)

- **Jaccard Distance**: we define the *Jaccard distance* of sets by $d(x, y) = 1 - SIM(x, y)$. That is, the Jaccard distance is 1 minus the ratio of the sizes of the intersection and union of sets x and y.
  We must verify that this function is a distance measure.
- **Cosine Distance**: We do not distinguish between a vector and a multiple of that vector. Then the cosine distance between two points is the **angle** that the vectors to those points make.
- **Edit Distance**: This distance makes sense when points are **strings**.
  The distance between two strings $x = x_1 x_2 \ldots x_n$ and $y = y_1 y_2 \ldots y_m$ is the smallest number of *insertions* and *deletions* of single characters that will convert x to y.

Example (Edit Distance)

The edit distance between the strings $x = abcde$ and $y = acfdeg$ is 3.
To convert x to y:

- Delete b.
- Insert f after c.
- Insert g after e.

No sequence of fewer than three insertions and/or deletions will convert x to y. Thus, $d(x, y) = 3$.

Example (Distances)

- **Hamming Distance**: Given a space of vectors, we define the Hamming distance between two vectors to be the number of components in which they differ. It should be obvious that Hamming distance is a distance measure.

Nearest Neighbour Classification (lazy learners)

The idea is to estimate the classification of an *unseen instance* using the classification of the instance or instances that are closest to it.

Example

Supposing we have a training set with just two instances such as the following

a	b	c	d	e	f	Class
yes	no	no	6.4	8.3	low	negative
yes	yes	yes	18.2	4.7	high	positive

There are six attribute values, followed by a classification (positive or negative). We are then given a third instance

yes	no	no	6.6	8.0	low	???

What should its classification be?

k-Nearest Neighbour or *k*-*NN* classification

Definition (Basic k-Nearest Neighbour Classification Algorithm)

- Find the k training instances that are closest to the unseen instance.
- Take the most commonly occurring classification for these k instances.

Issues:
There are several key issues that affect the performance of kNN.

- One is the choice of k. An estimate of the best value for k can be obtained by cross-validation.
- Another issue is the approach to combining the class labels. The simplest method is to take a majority vote. A more sophisticated approach, which is usually much less sensitive to the choice of k, weights each object's vote by its distance. Various choices are possible; for example, the weight factor is often taken to be the reciprocal of the squared distance: $w_i = 1/d(y,z)^2$.

Model Evaluation and Selection

Now that you may have build a classification model, there may be many questions going through your mind.

For example, suppose you used data from previous sales to build a classifier to predict customer purchasing behaviour.

- You would like an estimate of how accurately the classifier can predict the purchasing behaviour of future customers.
- You may even have tried different methods to build more than one classifier and now wish to compare their *accuracy*.

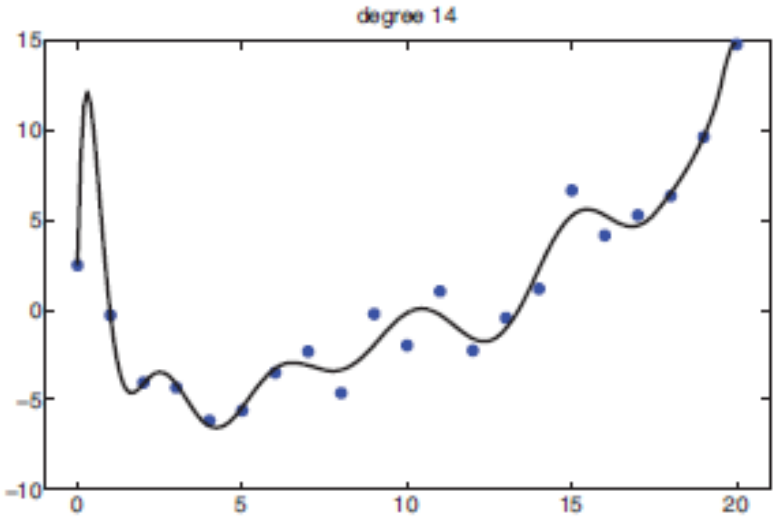
Questions:

- What is accuracy? How can we estimate it?
- Are some measures of a classifier's accuracy more appropriate than other?
- How can we obtain a *reliable* accuracy estimate?

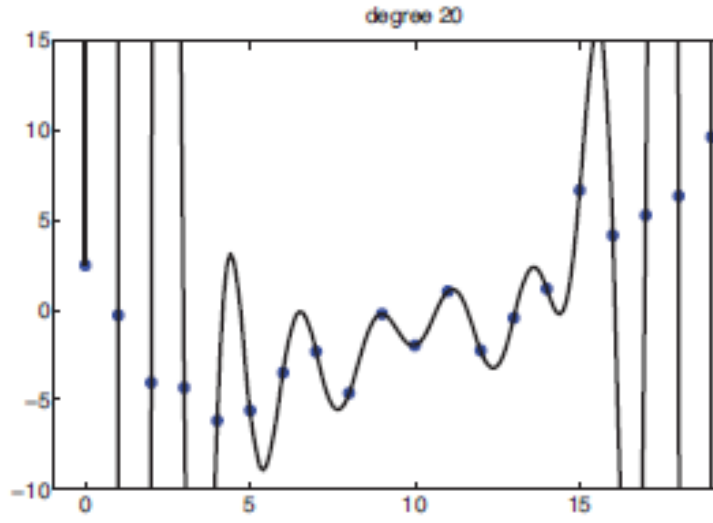
- The **misclassification rate** on the **training set**.

- However, what we care about is **generalization error**, which is the expected value of the misclassification rate when averaged over future data. This can be approximated by computing the misclassification rate on a large independent test set, not used during model training.

- **Overfitting**: When we fit highly flexible models, we need to be careful that we do not overfit the data, that is, we should avoid trying to model every minor variation in the input, since this is more likely to be noise than true signal. This is illustrated in Figure below, where we see that using a high degree polynomial results in a curve that is very "wiggly". It is unlikely that the true function has such extreme oscillations. Thus using such a model might result in accurate predictions of future outputs.



(a)



(b)

As another example, we will consider the *KNN* classifier. The value of K can have a large effect on the behavior of this model. When $K = 1$, the method makes no errors on the training set (since we just return the labels of the original training points), but the resulting prediction surface is very "wiggly". Therefore the method may not work well at predicting future data.

Classifiers and models depend on parameters.

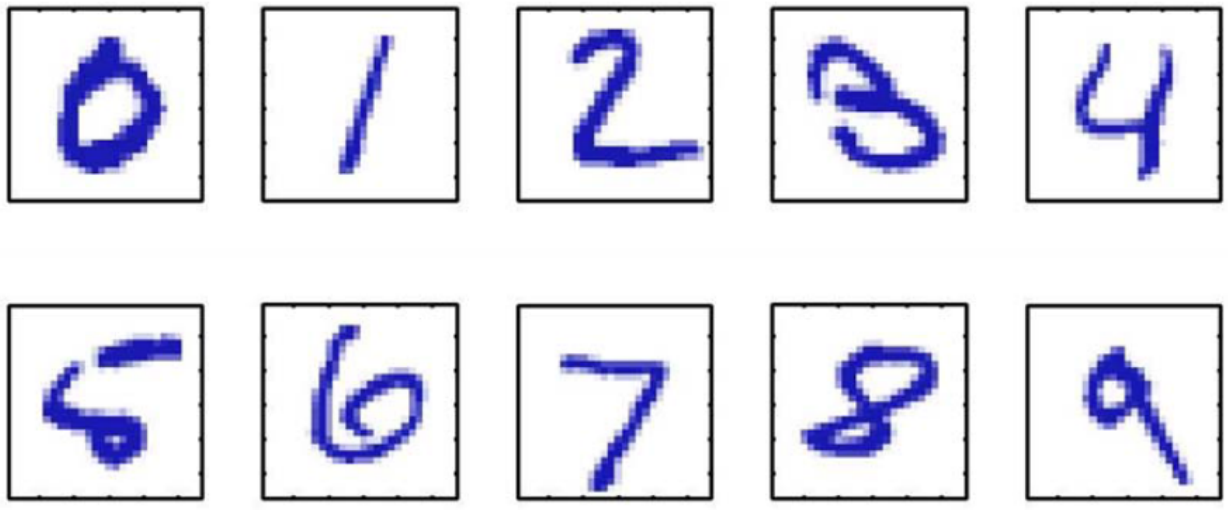
Typical:

60% - Training examples

20% - Cross-validation examples

20% Test examples.

Example 1: hand-written digit recognition



Images are 28 x 28 pixels

Represent input image as a vector $\mathbf{x} \in \mathbb{R}^{784}$

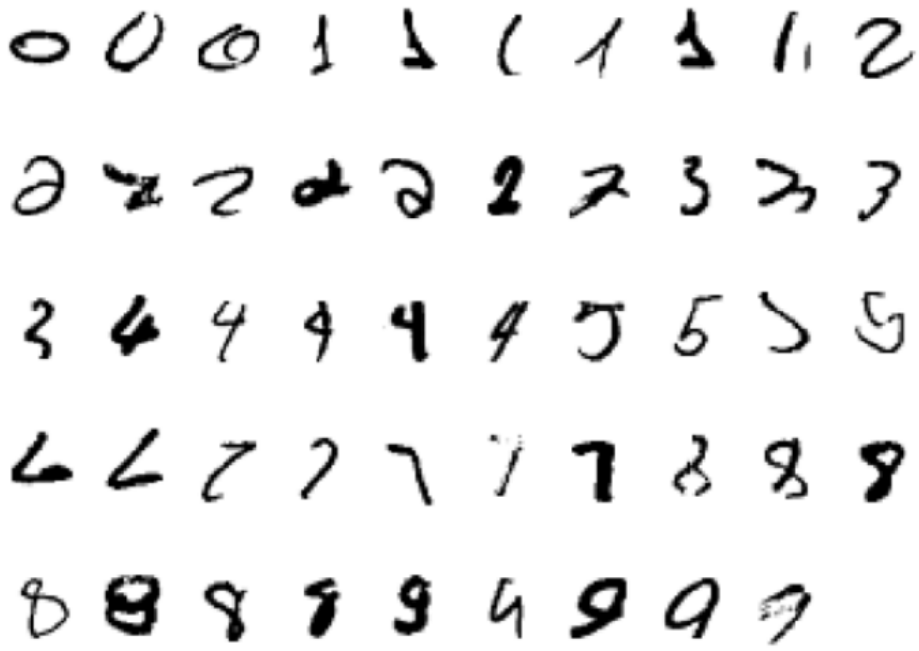
Learn a classifier $f(\mathbf{x})$ such that,

$$f : \mathbf{x} \rightarrow \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$$

How to proceed …

As a supervised classification problem

Start with training data, e.g. 6000 examples of each digit



- Can achieve testing error of 0.4%
- One of first commercial and widely used ML systems (for zip codes & checks)

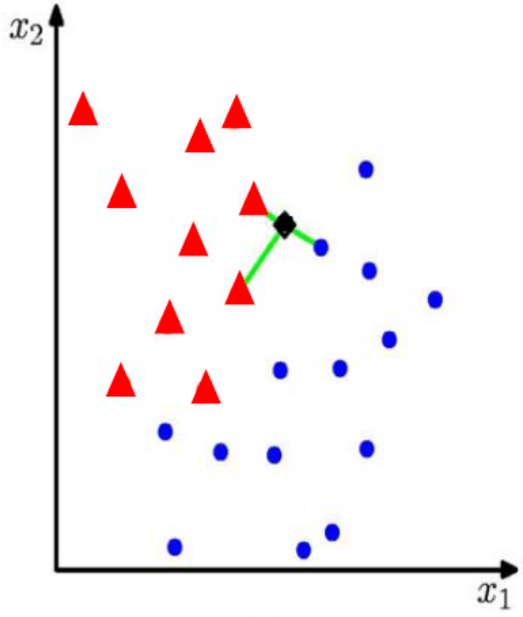
K Nearest Neighbour (K-NN) Classifier

Algorithm

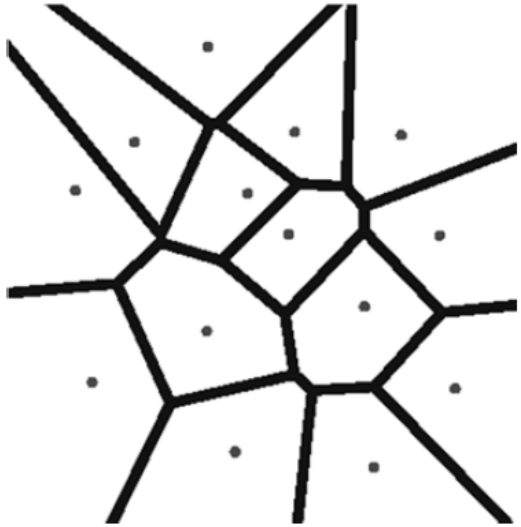
- For each test point, x, to be classified, find the K nearest samples in the training data
- Classify the point, x, according to the majority vote of their class labels

e.g. K = 3

• applicable to multi-class case

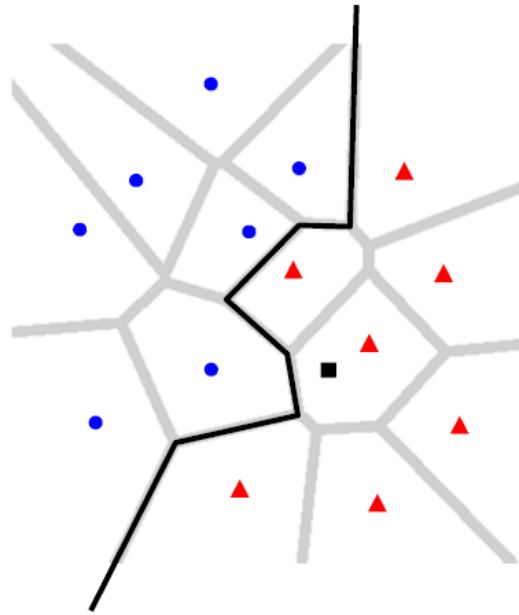


K = 1



Voronoi diagram:

- partitions the space into regions
- boundaries are equal distance from training points



Classification boundary:

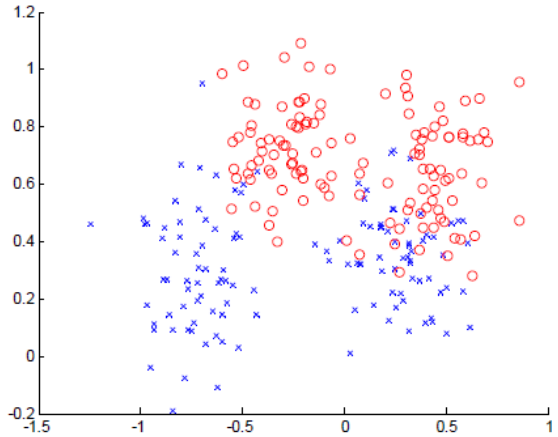
- non-linear

A sampling assumption: training and test data

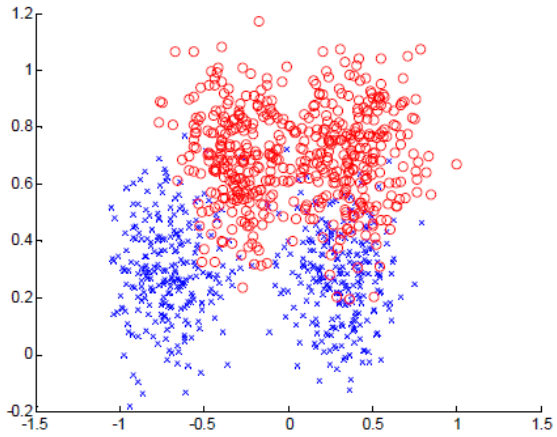
• Assume that the training examples are drawn independently from the set of all possible examples.

• This makes it very unlikely that a strong regularity in the training data will be absent in the test data.

• Measure classification error as $= \frac{1}{N} \sum_{i=1}^{N} \underbrace{[\mathbf{y}_i \neq f(\mathbf{x}_i)]}_{\text{loss function}}$ The "risk"



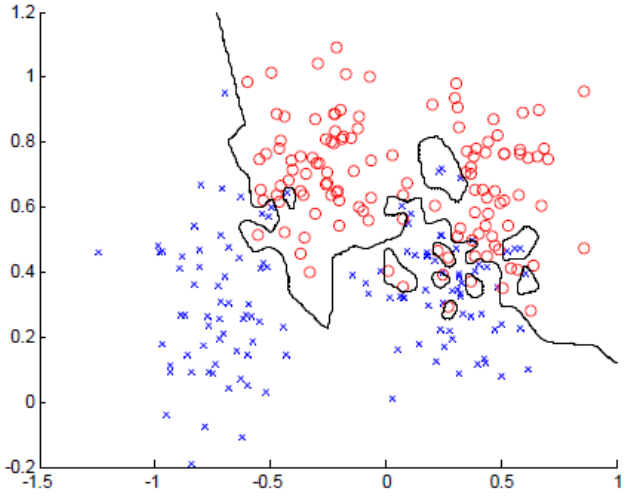
Training data



Testing data

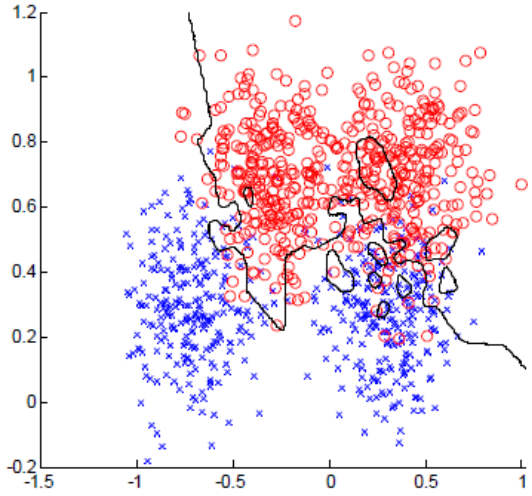
K = 1

Training data



error = 0.0

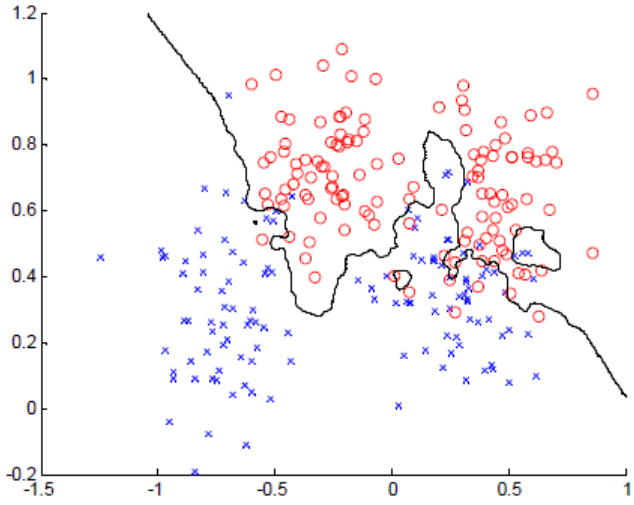
Testing data



error = 0.15

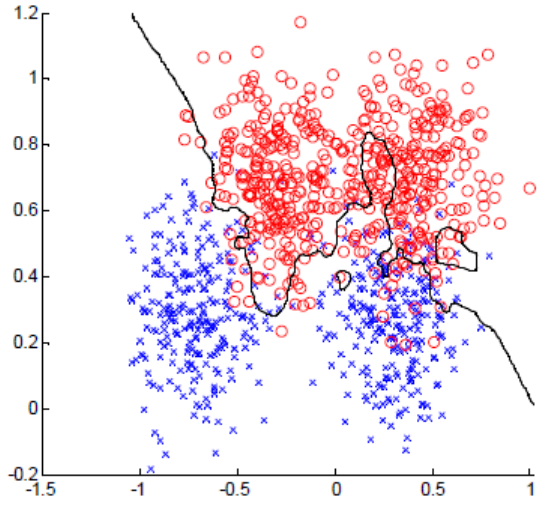
K = 3

Training data



error = 0.0760

Testing data



error = 0.1340

Generalization

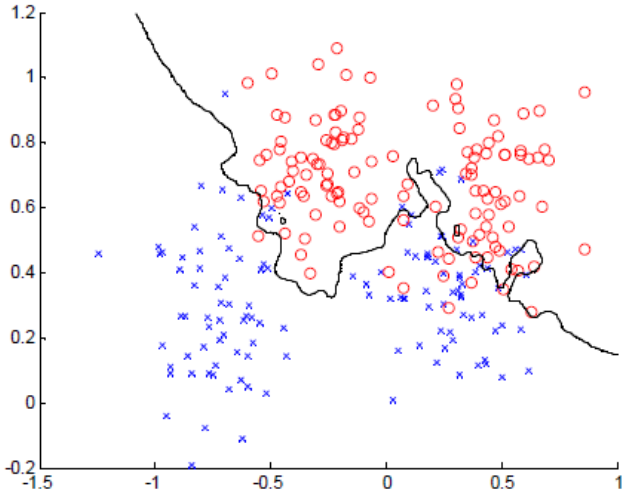
- The real aim of supervised learning is to do well on test data that is not known during learning

- Choosing the values for the parameters that minimize the loss function on the training data is not necessarily the best policy

- We want the learning machine to model the true regularities in the data and to ignore the noise in the data.

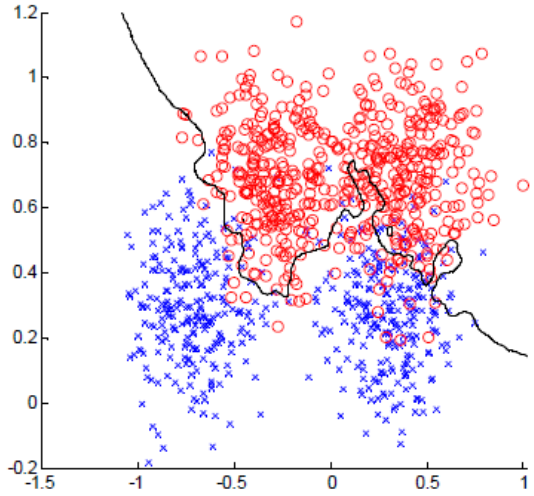
K = 7

Training data



error = 0.1320

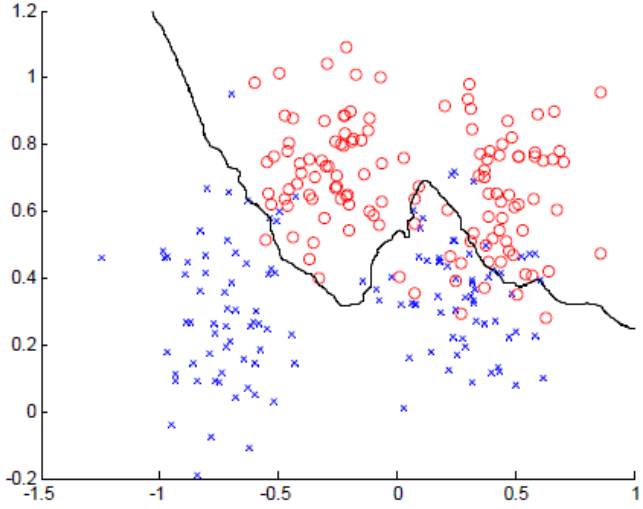
Testing data



error = 0.1110

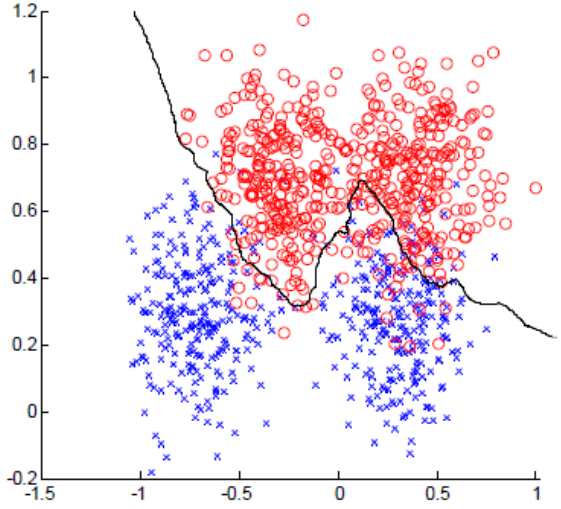
K = 21

Training data



error = 0.1120

Testing data



error = 0.0920

Properties and training

As K increases:

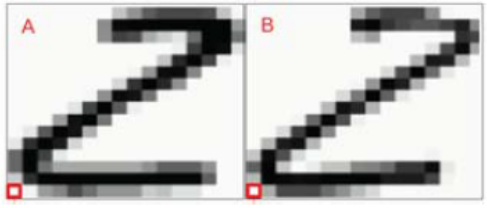
- Classification boundary becomes smoother
- Training error can increase

Choose (learn) K by cross-validation

- Split training data into training and validation
- Hold out validation data and measure error on this

Example: hand written digit recognition





- MNIST data set
- Distance = raw pixel distance between images
- 60K training examples
- 10K testing examples
- K-NN gives 5% classification error

$$D(\mathbf{A},\mathbf{B}) = \sum_{ij} \sqrt{\left(a_{ij} - b_{ij}\right)^2}$$

Cancer classification example

1. Train model

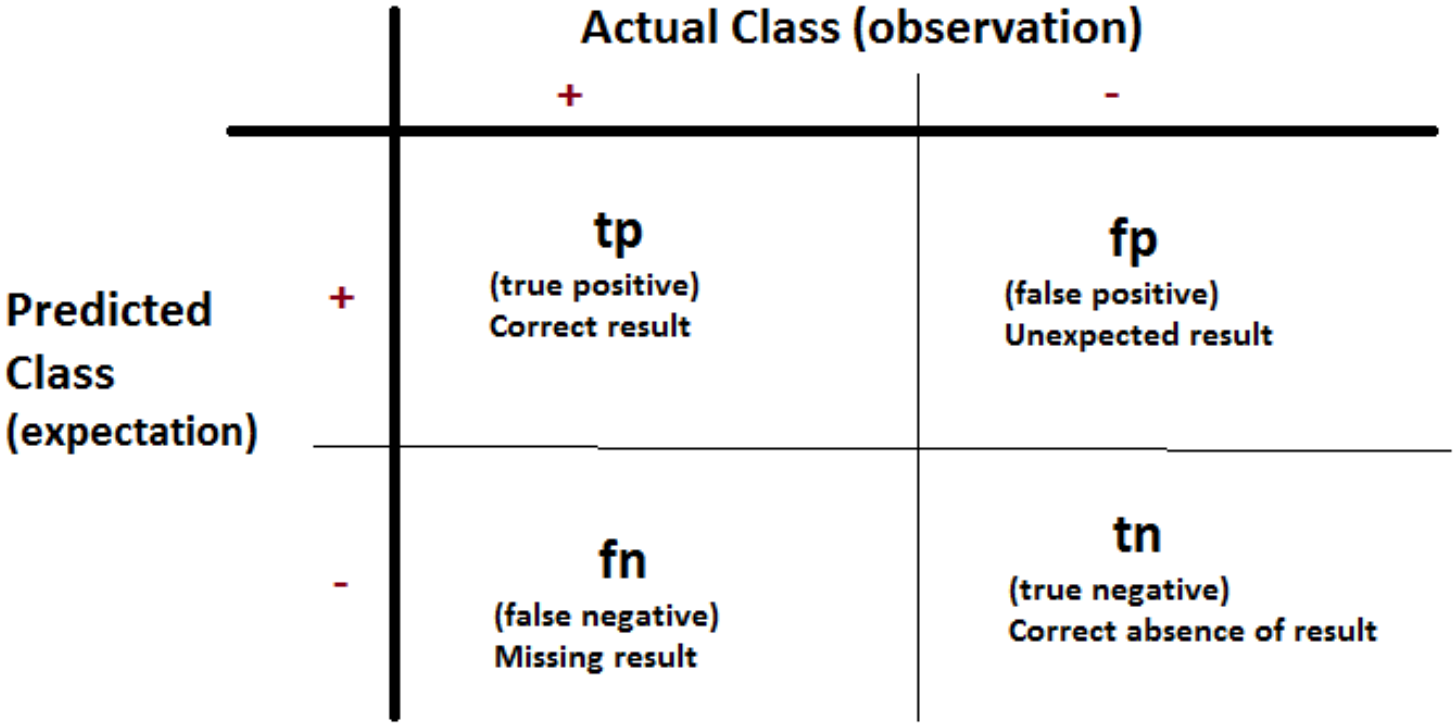
2. Find that you got 1% error on test set.(99% correct diagnoses)

3. Only 0.50% of patients have cancer (skewed classes).

4. y=0 has 0.5% error, 99.5% accuracy!

Precision/Recall

y=1 in presence of rare class that we want to detect.



Definition

$$\text{Precision} = \frac{tp}{tp + fp}$$

$$\text{Recall} = \frac{tp}{tp + fn}$$

$$\text{True negative rate} = \frac{tn}{tn + fp}$$

$$\text{Accuracy} = \frac{tp + tn}{tp + tn + fp + fn}$$

F-measure:

$$F = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

What are Bayesian classifiers?

Bayesian classifiers are *statistical classifiers*. They can predict class membership probabilities such as the probability that a given tuple belongs to a particular class.

Bayesian classification is based on Bayes' theorem.

Naïve Bayesian classifiers assume that the effect of an attribute value on a given class is independent of the value of the other attributes.

This assumption is called class-conditional independence.

- Let **X** be a data tuple.
  As usual, it is described by measurements made on a set of n attributes.
- Let H be some hypothesis such as that the data tuple **X** belongs to a specified class C.
  For classification problems, we want to determine $P(H|\mathbf{X})$, the probability that the hypothesis H holds given the "*evidence*" or observed data tuple **X**.

Bayes' Theorem

$$P(H|\mathbf{X}) = \frac{P(\mathbf{X}|H)P(H)}{P(\mathbf{X})}.$$

Naïve Bayesian Classification

The **naïve Bayesian** classifier, or **simple Bayesian** classifier, works as follows:

1. Let D be a training set of tuples and their associated class label.
   As usual, each tuple is represented by n-dimensional attribute vector, $\mathbf{X} = (x_1, x_2, \ldots, x_n)$, depicting n measurements made on the tuple from n attributes, respectively, $A_1, A_2, \ldots, A_n$.
2. Suppose that there are m classes, $C_1, C_2, \ldots, C_m$.
   Given a tuple, $\mathbf{X}$, the classifier will predict that $\mathbf{X}$ belong to the class having the *highest posterior* probability, conditioned on $\mathbf{X}$. That is, $\mathbf{X}$ belongs to the class C_i if and only if

$$P(C_i|\mathbf{X}) > P(C_j|\mathbf{X}), \quad \text{for } 1 \leq j \leq m,\ j \neq i.$$

Naïve Bayesian Classification

3. By Bayes' theorem, $P(C_i|\mathbf{X}) = \frac{P(\mathbf{X}|C_i)P(C_i)}{P(\mathbf{X})}$.
   As $P(\mathbf{X})$ is constant for all classes, only $P(\mathbf{X}|C_i)P(C_i)$ needs to be maximized.
4. **The class prior probabilities**:
   - If the class prior probabilities are not known, then it is commonly assumed that the classes are equally likely, that is, $P(C_1) = P(C_2) = \cdots = P(C_m)$, and we would therefore maximize $P(\mathbf{X}|C_i)$.
   - Otherwise, we maximize $P(\mathbf{X}|C_i)P(C_i)$. Note that the class prior probabilities may be estimated by

$$P(C_i) = \frac{|C_{i,D}|}{|D|},$$

   where $|C_{i,D}|$ is the number of training tuples of class C_i in D.

Naïve Bayesian Classification

5. How to compute $P(\mathbf{X}|C_i)$?

To reduce computation and complexity in evaluating $P(\mathbf{X}|C_i)$, the naïve assumption of **class-conditional independence** is made.
This presumes that the attributes' values are conditionally independent of one another, given the class label of the tuple (i.e. that there are no dependence relationship among the attributes). Thus

$$P(\mathbf{X}|C_i) = \prod_{k=1}^{n} P(x_k|C_i)$$

$$= P(x_1|C_i) \times P(x_2|C_i) \times \cdots P(x_n|C_i).$$

We can easily estimate the probabilities $P(x_1|C_i), P(x_2|C_i), \cdots P(x_n|C_i)$ from training tuples.

(a) If A_k is categorical, then $P(x_k|C_i)$ is the number of tuples of class C_i in D having the value x_k for A_k, divided by $|C_{i,D}|$, the number of tuples of class C_i in data D.
If the count is zero, we can use the **pseudo-count** method to obtain a prior probability. The adjusted estimates with pseudo-counts are given as

$$P(x_j|C_i) = \frac{n(x_j) + 1}{n_i + m_j},$$

where $m_j = |dom(A_j)|$.

(b) If A_k is continuous-valued, we need to do a bit more work: parameter estimates.